University of Glasgow | School of Computing Science

Investigating Logical and Empirical Minesweeper Field Difficulty with Constraint Programming Techniques

Andrei Boghean
2645295b

Abstract

Minesweeper is an NP-complete puzzle game whose random board generation can yield very different boards with drastically different solve difficulties, even within the same fixed board parameters, despite the simplicity of board generation. Due to the unpredictability of this variance, there does not currently exist a difficulty metric that can capture it. Instead, players rely on metrics such as solve time and minimum clicks necessary, which are believed to be insufficiently representative.

By looking at Minesweeper as a constraint satisfaction problem, we apply well-established algorithms, namely Relational Arc Consistency (RAC), to objectively categorise the difficulty of different Minesweeper moves, and subsequently apply this to both the moves needed to solve a board and the moves used by a player, in an effort to understand how these moves influence the time to solve the board.

We find that, while our approach is highly limited by its dependence on existing playthrough recordings on the field, it still demonstrates that aggregating RAC is viable for estimating the field difficulty, and may eventually be used to produce an accurate automated estimate if the human recordings can be effectively replaced with machine-produced solves instead. In addition, we also look at RAC as an analysis tool for human ability and conclude that it shows potential for quantifying a player's mastery of techniques, provided there is enough data on that player, but suffers from ambiguity within classes of moves as plain RAC only identifies the number of constraints involved in solving a cell, rather than the specific pattern used.

1 Introduction

Minesweeper is a notable constraint satisfaction puzzle game which has gained significant popularity since its invention, and has developed a strongly competitive community who compete to hold the fastest solve time over random fields of standard sizes and mine counts. Field sizes were kept fixed as an early attempt to anchor the difficulty of the field, but Minesweeper fields can still vary drastically in difficulty even within the same field size (much like other constraint satisfaction puzzles such as Sudoku or Sokoban as shown by Pelánek [22] and Jarus˘ek and Pelanek [15] respectively). In additional attempts to standardise field difficulty, other metrics for fields were invented, but no metric yet can predict the logical difficulty of fields sufficiently to quell debates within the competitive scene such as the one studied by de Winter [9].

The class of constraint satisfaction puzzles, containing Minesweeper, is known to be NP-complete. NP-complete problems are a well-studied category, with advanced modelling languages such as MiniZinc (Nethercote et al. [20]) that run on complex solvers which are both highly optimised and rigorous in their operation. On this note, the problem of describing field difficulty in Minesweeper stands to benefit greatly from work on NP-complete problems.

Our aims start with understanding existing research relating to the difficulty of NP-complete problems, both in the purely logical sense and in the human-experienced empirical difficulty. We then seek to apply this to Minesweeper fields to better understand what influences the difficulty of a field, and eventually produce a well-informed estimate for the logical difficulty. In addition we also seek to study the human factor and how human play changes across players, with respect to the player's demonstrated logical ability.

The structure of this document begins with background information on the basics of Minesweeper, the logical complexity of Minesweeper, and then a deep dive into various research relating to the difficulty of NP-hard problems. We then collate this research into an broad model for the difficulty of a Minesweeper field, and immediately go into compromises necessary to reach a feasible version that can be implemented and tested. In evaluation, we first lay out both our primary and auxiliary results in difficulty aggregation and human-skill insights respectively. Secondarily, we discuss these results, their implications, and their trustworthy-ness. Lastly, we conclude on the general sentiment from our results, and lay out future work to bring this project closer to a difficulty estimation effective enough to succeed 3BV.

2 Background

To establish the background information needed for this project, we first explain the basics of Minesweeper, the player's needs for self-improvement in the game, and why there is an unmet need for a sufficiently accurate metric to rank the difficulty of Minesweeper fields.

With this, we then explain the nature of Minesweeper as an NP-hard problem to provide the logical basis for our algorithm design, and then explore similar research into CSP difficulty and other adjacent areas for insights we may apply ourselves. Lastly we focus on a particularly straightforward algorithm for ranking the plain computational difficulty of individual Minesweeper techniques, which will eventually serve as the basis for our difficulty estimation after extending it with difficulty factors explored by other research.

2.1 The Basics of Minesweeper

Minesweeper is a popular logic game which was first released in 1989 as part of Microsoft Windows. It is a grid-based logic game, where the player is presented with a finite board featuring covered cells that may contain either a mine or an empty space. The player is tasked with uncovering all empty spaces on the field, usually done by deducing non-empty cells via the numbers inscribed on each cell stating the number of mines in the surrounding 8 cells.

The field generation can feature arbitrary width, height, and mine count, but most play focuses on the standardised difficulties of Beginner (shown in Figure 1), Intermediate, and Expert. These difficulties correspond to boards of 9 by 9 with 10 mines, 16 by 16 with 40 mines, and 30 by 16 with 99 mines (respectively).

In addition to the basic logic, the standard game also allows the player to keep track of mines by flagging suspected cells with mines, allows the player to execute a "chord" (explained below), and further helps them by automatically uncovering neighbours of a cell inscribed with 0.

"*Chording*" is a feature, mainly serving as a convenience, which will automatically clear all neighbouring (both immediate and diagonal) cells that are covered and un-flagged, provided the hint inscribed on the relevant cell matches with the number of neighbouring flagged cells, i.e. all neighbouring mines are marked.

This feature is typically triggered by pressing both the left and right mouse button simultaneously (two simultaneous actions, akin to a musical *chord*) and is mostly used to save time and clicks when all cells are found in a 3x3 area. It is typically counted as the actual amount of mouse button clicks, meaning that there are fewer clicks counted compared to manually opening the neighbouring, known safe, cells.

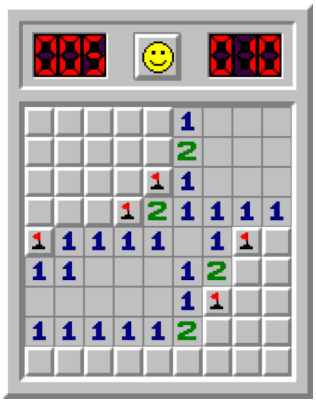


Figure 1: A partially completed beginner board showing covered cells, covered cells flagged by the user, and opened cells with hints 0-8 indicating the number of neighbouring (both immediate and diagonal) mines.

2.2 Minesweeper Self-Improvement

To justify and subsequently guide our research on an accurate difficulty metric for Minesweeper fields, we first cover the fundamental need for such a metric and why this need is believed to be insufficiently met by the existing difficulty metrics.

2.2.1 The Need for an Accurate Difficulty Metric

As is covered across prior work such as that by [12, 13, 18], people are generally incentivized to maximise their "reward per unit of time invested". The effective reward per unit time is an abstract concept, influenced by the player's problem solve time (determined by their skill), and the reward of the problems they're completing. To maximise this metric the player then aims to solve as many problems as they are able to, which generally also means maximising problem difficulty, since the two are frequently correlated. When the player's ability appropriately matches the complexity of the problems they're facing, they're said to be in a "flow" state, where the player is believed to be operating at their best level and displaying maximal performance.

Engeser and Rheinberg [12] outline additional specific conditions for flow state, which are generally met during Minesweeper gameplay, but two conditions specifically are relevant for consideration here: 1. there is a balance between a user's perception of their skills and the offered challenge, and 2. the offered challenge is clear and unambiguous.

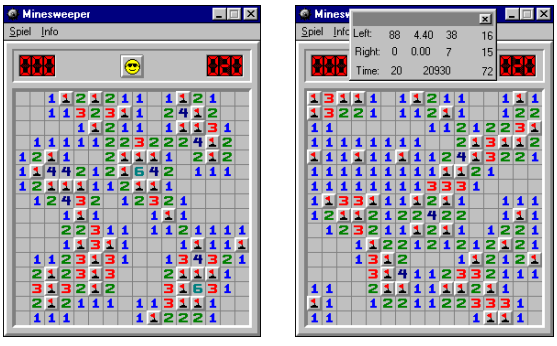
The second point is what motivates a player's need for an accurate difficulty metric. It is necessary not just for the player to select the difficulty they're facing to practice "flow", but is also useful in comparing performance across different fields by the same player.

2.2.2 Available Difficulty Metrics and Their Flaws

As established, dedicated players aim to incrementally improve their skill subject to some metric indicative of their performance. In Minesweeper, there are two main metrics used in this process. The primary metric is solve time. For a player, solve time primarily serves as an objective reward which indicates how skilful the player was, and lets them adjust their methods accordingly over multiple plays to optimise their performance.

Solve time is not influenced just by player ability, but also by field difficulty. When evaluating their play with the aim to improve, the player needs to "normalise" the performance they were expecting to display, to adjust for the field's difficulty. This is why players typically try to improve over randomised placements of a fixed number of mines within a field of a fixed size, since it reduces difficulty variance. Games generated subject to these conditions still do not necessarily have the same difficulty over different randomisations, as different configurations can produce sequences of mines that might force more guesses, require moves of varying difficulty, bias speed of travel around the grid, capacity to memorise mines, pattern recognition, *etc* (these ideas will be explained more concretely in Section 5).

In an effort to further mitigate this, Bechtel [4] created the "3BV" score, a metric which simply counts the minimum number of necessary clicks to solve a board without using any flags. This is calculated upon completion of the board, and is used retrospectively by the player to adjust their belief on the quality of their performance. This is still insufficient by some accounts, as is acknowledged by the creator of the aforementioned 3BV, who presents as an example two fields they had solved themselves in the same time (20s) with widely differing 3BV scores (38 and 72) shown below in Figure 2. The general sentiment in the wider competitive Minesweeper community is similarly critical of its usefulness as a difficulty indicator on its own, as can be seen in some of the many online forum discussions on the topic[5, 17, 19].



(a) 20s field with a 3BV of 38 **(b) 20s field with a 3BV of 72**

Figure 2: Fields solvable in 20s with drastically different 3BV scores

2.3 Problem Statement

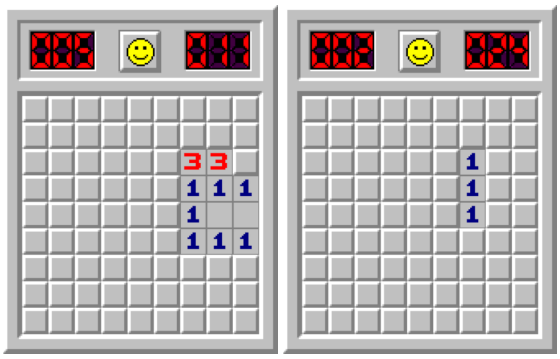
So far there exists no metric or combination of metrics that can sufficiently capture the nuances in difficulty of Minesweeper boards, with players instead resorting to a combination of surface-level metrics for an educated estimate.

Considering Minesweeper's nature and its difficulty as an NP-hard game, as shown by Kaye [16] whose argument we will explore below, we aim to explore the possibility for a CSP approach to provide a deeper insight into the difficulty of a Minesweeper field and allow for more accurate estimation beyond the metrics explored, while also taking into account human factors and investigating how player skill relates to the expected "ideal" difficulty.

2.4 Logical Complexity of Minesweeper

The computational complexity of Minesweeper is well understood to be NP-hard. This fact was first established by a very popular paper by Kaye [16], in which the author simplified Minesweeper to what will be referred to as the "Minesweeper Consistency" problem. This consistency problem is a sub-problem found in Minesweeper, that is simpler than playing the whole game but still complex enough to capture the "core" difficulties of Minesweeper's computational complexity. Kaye [16] defines Minesweeper consistency as a problem operating over "a rectangular grid, partially marked with numbers and/or mines, some squares being left blank" which asks whether there exists an assignment of mines to covered cells which is consistent with the numbers revealed in open cells (i.e. all cells numbered x have exactly x neighbouring mines). To prove consistency is an NP-complete problem, Kaye [16] begins by establishing that Minesweeper is in NP, by showing it is possible to transform a Minesweeper field into a boolean satisfiability problem. He states this is done by isolating an arbitrary 3 by 3 grid of cells with a cleared middle cell stating n neighbouring mines, and argues this is equivalent to a boolean satisfiability problem where each clause is a different possible arrangement of mines among the neighbours, with the full 3 by 3 sub-grid being a disjunction of the possible $\binom{8}{n}$ hidden mine combinations. Kaye [16] then assembles these individual SAT problems for 3 by 3 sub-grids into a full statement for the whole board via conjunction. To provide a polynomial time reduction from an NP-complete problem to Minesweeper, Kaye [16] first shows that it is possible to create boolean logic circuits in Minesweeper by showing how to create a simple wire via a mine arrangement where a designated output cell will contain a mine if and only if a designated input cell will contain a mine. Kaye [16] then extends this with wire bends and junctions, and finally boolean gates NOT and AND, which provides the basis for the other fundamental logic gates OR and XOR. Using this digital computer, Kaye [16] shows how, for any instance of boolean satisfiability, a Minesweeper circuit can be constructed where solving for consistency also solves the original SAT instance. Having produced a proof for Minesweeper consistency being NP-complete, Kaye [16] then went on to generalise this sub-problem to the whole of Minesweeper, eventually concluding that a typical play through of the game is also NP-complete.

This conclusion is almost as widely accepted as their NP-completeness proof for Minesweeper consistency, but there is some disagreement. Scott et al. [24] disputes Kaye [16]'s generalisation of NP-completeness from consistency to the game as a whole, on the basis that Kaye [16] did not consider the possibility that their proposed algorithm for solving general Minesweeper via application of the consistency problem is not necessarily the most efficient method. To explain their objection, Scott et al. [24] introduces a sister sub-problem to Minesweeper consistency - that of Minesweeper inference. This problem takes as input a board similar to consistency, with open cells, closed cells, and flags, and asks whether it is possible to make progress on the given board by identifying a cell as safe or containing a mine, subject to the condition that the board is already consistent and all flags are correct. Intuitively, this asks whether the only remaining actions on the field all force the player to guess, examples for which are shown in Figure 3. Scott et al. [24] demonstrates that the inference problem is in fact a co-NP problem, and subsequently presents a reduction from the unsatisfiability problem (the co-NP-complete variant of SAT) to Minesweeper inference through a very similar method to Kaye [16] - a general reduction for any (un)SAT problem to a computational circuit of logic gates, constructed through the rules of Minesweeper, that solves the respective problem when consistency is solved on the reduced Minesweeper problem.



(a) Field progressable without guessing **(b) Field progressable only with guessing**

Figure 3: partially solved fields that are progressable with and without guessing

2.4.1 Consistency/Inference Relation

Through reasoning about the conditions for a Minesweeper move to be possible, we want to highlight that Inference itself can also be defined in terms of consistency.

When a player is attempting to solve Inference through playing, a decision on a cell can only be made confidently when it is unambiguously the only possible result for that cell. This means that, in all possible solutions to Consistency, that same cell takes the same result in any and every case. In other words, Inference can be seen as asking whether the number of unambiguous cells across all Consistency solutions is greater than 0. If there are multiple unambiguous cells, then there are multiple Inference solutions.

For some examples, consider the special case where there is only one solution to Consistency. Since no cell is ambiguous, Inference is possible on every cell, and the field is fully solvable. On the other hand, in a field where Inference is not possible anywhere, that must mean there is no hidden cell with an unambiguous result, meaning every undecided cell has multiple possible results across all consistency solutions, i.e. there is always ambiguity. In Consistency terms, that means there are either no solutions or more than two.

While these problems do not define a full Minesweeper instance, a full solve can be broken down into a repeated application of Inference, where progress made on an individual instance of inference implicitly keeps the field "consistent". These individual steps still comprise NP-hard problems, so our research into difficulty will need to focus on Inference and develop methods to aggregate the difficulty over the individual occurrences that eventually make the full solve of a Minesweeper field.

2.5 Influencing Difficulty by Manipulating CSP Strategies

Knowing Minesweeper can be classified as a constraint satisfaction problem, we can look to other studies investigating the difficulty of Constraint Satisfaction Puzzles, for insights that might extend to Minesweeper. Specifically, we look at research in sourcing Minesweeper heuristics, applying them, and lastly trying to manipulate them, in order to demonstrate the link between community-learned heuristics (patterns), and the perceived difficulty in the CSP game.

2.5.1 Sourcing Common Game Strategies

Before we can look at CSP strategies and how they relate to difficulty, we first need a way to identify strategies in the first place for investigation. In Pelánek [22]'s research on encoding sudoku techniques into a computational model, they rely solely on techniques learned and documented by players through gameplay (specifically - techniques documented in popular puzzle solving books). While such a source may be non-exhaustive, it serves as a good starting point and likely includes most of the frequently encountered techniques.

2.5.2 Research Applying Community-Documented Game Strategies

In Minesweeper, there have been attempts to implement both classical (i.e. intuition or heuristics based) algorithms and constraint-based models, with some of these attempts basing themselves on previously identified human strategies and behaviours.

One such "classical" attempt is the "Single Point" strategy. The concept behind the single point strategy (SP) centres around the most common and simplest inferences or patterns that Minesweeper players first learn, such as the one shown in Figure 5a. The technique was formalised into SP strategy by Pedersen. Execution of SP starts by establishing an internal set of known valid moves, which the player picks moves from and executes. Upon execution of a move, the set is refreshed with newly identified safe moves. Safe move identification is based on the simplest Minesweeper technique - the identification of cases where the number in a cell is exactly equal to the number of neighbouring flagged cells which allows all neighbouring closed cells to be revealed, and secondly the identification of cases where the number of unopened neighbours is equal to the count of a cell, allowing the player to conclude that all neighbouring cells contain mines.

Becerra outlines a problem with the SP strategy where squares that did not immediately yield useful information are discarded prematurely. To mitigate this, they introduce a "Double Set Single Point" strategy. This strategy is defined as an extension to the simpler single point algorithm. Where SP would discard a square on a Minesweeper field that does not fall into either of the 2 conditions for "progress", DSSP instead adds that square to a secondary set Q, that the algorithm returns to once it has exhausted all first-order squares with immediately accessible information for progress.

2.5.3 Research Manipulating General Game Strategies

In general game design, Reis et al. [23] focuses on combining mini levels together to form cohesive larger levels and create a "storyline" in a mario level. This is done by applying "Tension Arcs", which organise the order in which sub-levels are combined to maintain a smooth ramp-up in difficulty to a peak in the middle of the level which then starts to decay as the level finishes (effectively, the "story").

This notion of "difficulty regions" is also important in CSP games, for example in Ansótegui et al. [1]'s research where they use the concept to get around scaling sudoku boards beyond sizes of 5 blocks (a block being a 3 by 3 sub-section), where the difficulty of solving increases drastically.

To progress difficulty further while not having to scale the board size, they research board generation methods which "hide" complex combinations of constraints in generated fields to control the flow of difficulty. Their approach starts with a given, complete, sudoku field, and creates permutations via specific permissible actions that rearrange the board in fashions which do not violate the rules. They apply difficulty controls as rulesets for this generator, with the core of their difficulty control being the "balance" of empty cells through rows, columns, and blocks. Their difficulty balancing uses a network representation of sudoku boards, in which there is $n + m$ nodes for each n rows and m columns with nodes being connected if and only if that (row, column) position on the board contains an empty cell. Using this framing, they constrain their generator to enforce a regular bipartite graph, which effectively "balances" empty cells between rows and columns so there does not exist any disproportionately weighted node. Maintaining a node balance then ensures that empty squares are spread out and interact with each other minimally, so the user is only able to make slow consistent progress rather than solving a single empty square for a significant chunk of the board.

This balancing approach via a regular bipartite graph addresses the easiest method for progress in sudoku - filling out empty cells that are alone in an otherwise filled out row/column/block. Sudoku, much like Minesweeper and other logic games, has more complex techniques available to a player beyond the techniques mitigated by these surface-level balancing operations. While the demonstrated difficulty balancing only balances the usability of easier techniques, it brings up an important idea of "technique reward", whose average value and variance could impact the reward in a given map for luck or a gambling-focused play style.

2.5.4 Heuristic-Based Examples in Categorising Field Difficulty

Getting closer to Minesweeper, Cicvárek [7] creates a Minesweeper board generator, and much like Pelánek [22] they also source field candidates via random placements. In their case however, presumably since Minesweeper boards are much more relaxed in their rules for "consistency", Cicvárek does not need to use a complex board generator or permutation scheme. Instead, their approach constructs a board by randomly placing mines in an empty grid and using Minesweeper solver algorithms to validate that inference is possible, proposing that this should guarantee the whole field is solvable.

Their sequence of algorithms for validation starts with single point, then a backtracking algorithm with complex guessing, then an extension to single point with a linear equation solver, then a good-enough polynomial-time solver, and finally an exhaustive algorithm with exponential complexity.

This generator approach, using a wide selection of different solving options with increasing complexity, was found to be suitable at categorising fields with a "rank" metric (the largest number of variables ever needed to consider in a single step in the whole field solve), ranging up to rank 9. This is useful for basic difficulty estimation, but using the maximum observed difficulty can be fairly misleading, for example on a Minesweeper field that contains a single rank-9 move among a majority of rank 1 moves. In comparison with a field full of rank 5 moves, the former shouldn't be considered harder by a suitable difficulty ranking. In addition, much like the balancing phenomenon covered in the previous section, the difficulty of a portion of a Minesweeper field can change drastically according to the direction the field is approached from, and what adjacent portions have already been solved. These nuances demand that multiple estimations for the same field are considered from a wide variety of pathways, unless the field happens to be balanced by chance through the random generation.

2.6 Applicable Strategies Can Be Found by Relational Arc Consistency

The approaches explored so far, which aim to understand and control difficulty, demonstrate the important idea of "local difficulty" or "strategy biases" within their given problem. That is, local portions of a problem instance that necessitate "harder" techniques to solve.

We might then want to look at similar strategy biases within Minesweeper fields, to get a more granular understanding of how difficulty is affected. A method for doing this is already partially demonstrated by Bayer et al. [2], who aims to create a tool, via constraint programming, to assist players in solving Minesweeper fields. To do this, they give the player the option to ask the computer to apply varying levels of the relational arc consistency algorithm $RAC_{(i,m)}$, as defined by Dechter [10], for three different levels $RAC_{1,2,3}$ (with parameter i being left unbounded and ignored), and either solve all moves revealed by the algorithm or simply identify them.

Their use of Relational Arc Consistency inadvertently demonstrates how it can relate to Minesweeper strategies of varying difficulty, as can be seen in Figure 4. While their paper does not thoroughly study this relation, their demonstration still supports the idea and provides a good starting point in investigating how constraint satisfaction techniques relate to game solving heuristics learned by humans.

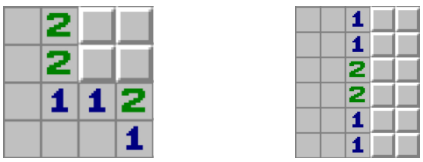
2.6.1 Relating RAC to Minesweeper Techniques

The connection between RAC and Minesweeper can be explained by looking at some basic Minesweeper techniques, and running through the logic by hand. RAC_1 can generally be described by cases where the number of unrevealed neighbouring cells equals the number inscribed as the hint of a cell. It can be demonstrated by an easy corner pattern, as shown in Figure 5a. Simply, the 1-hint on the "corner" of the structure of unrevealed cells indicates exactly one mine in its vicinity. Since there is only one unrevealed cell, it must be there, so that single corner cell can be noted as containing a mine.

Figure 4: Minesweeper field annotated with blue, green, and yellow illustrating possible progress discernable via *RAC* with parameter m being 1, 2, and 3 respectively.

RAC_2 can be demonstrated by looking at the 1221 mine arrangement (shown in Figure 5b) as an example. It features the hints 1,2,2,1 against a wall of closed cells. Recall RAC_2 means that two constraints are being considered in combination. The first decision we can make on this arrangement uses one of the two 2-hint cells, where this example will be using the top one. It tells us that there is two mines in the three hidden cells to the right of the 2-hint. This is still ambiguous, since there are 3 different possible arrangements for the two mines among the 3 hidden cells. As the second constraint, we can pick the 1-hint above the 2-hint (note that RAC wouldn't "pick" a constraint, but simply check every possible combination and go with whichever does not violate consistency). This 1-hint tells us similarly that there is one mine within the three hidden cells to the right. There is an overlap of two hidden cells between the 1-hint and the 2-hint. The 1-hint requires there to be a max of one mine in all three of its neighbours, which also implies that there is a max of one mine in this smaller group of 2 cells. In the context of the 2-hint, this means there is at most 1 mine in the area which overlaps with the 1-hint, and there is $2 - 1 = 1$ mine in the neighbouring area of the 2-hint which contains the remainder of the mines. Since there is only a single cell in the remaining area, it must be the cell containing the mine. This conclusion then allows the 1-hint to have its other neighbours opened safely, which removes ambiguity in the area common between the 1-hint and the 2-hint, making it finally clear that both cells immediately to the right of the 2-hint cells both have mines, and the 1-hints do not have any mines to their right.

RAC_3 is more complex to communicate in this manner and not worth the verbosity necessary to explain sufficiently, but could be understood by inspecting the yellow cells in Figure 4.



(a) Easy corner pattern **(b) Slightly hard 1221 pattern**

Figure 5: Minesweeper patterns of different difficulty

2.7 Background Summary

Minesweeper can be viewed from different perspectives given by two distinct but related sub-problems, Consistency and Inference, which place Minesweeper within the class of NP-hard puzzle games. This allows us to apply insights from other NP-hard puzzle games and problems, notably efforts in solving, manipulating, and detecting local sub-problems of different difficulties via either heuristic or exhaustive algorithms. From this review we find Relational Arc Consistency to be useful in labelling cells that are solvable via Minesweeper Inference, according to the level of RAC necessary for solving the cell.

Motivated by this, we proceed with RAC as tool for labelling cell difficulty, and explore methods for combining the difficulty of individual cells to obtain an estimation of the whole field's difficulty rather than its constituent parts. Simultaneously, we apply our adjacent research to lay out a model for the human process of playing Minesweeper to serve as a reference while exploring methods of cell difficulty aggregation.

3 Methodology

In our methodology section we start by reviewing potential difficulty influences from our background, to build an overall understanding of the end-to-end difficulty in a Minesweeper playthrough, and then outline a selection of compromises on this theoretically all-encompassing model that lead us to a smaller but feasible proof of concept and the actual implementation we evaluate.

3.1 Modelling the Human Process of Playing Minesweeper

As we've explored, the effective difficulty of a Minesweeper field as experienced by the player is very complex, with many contributing factors. This subsection distils these influences into an overarching model that aims to encompass enough factors to properly replicate the final end-to-end difficulty when a human is solving a Minesweeper field.

Broadly, our model categorises contributing factors of Minesweeper field difficulty into 3 groups:

(1) contributors from the logical complexity of the field.
(2) aspects from the player's ability and experience.
(3) random influences from the player's arbitrary pathfinding around the Minesweeper field.

3.1.1 Minesweeper Field Structure Arising From Logic

As a symptom of Minesweeper's nature, playing a large enough game with an appropriate mine density can give rise to "clusters". These are defined, by Cicvárek and Kopřiva [8], to be groups of cells where each constraint on each cell operates on other cells who are also part of this cluster, i.e. there is no logical "connection" between clusters; solving one cluster never yields any information on another.

Thanks to this game structure, players can and do deviate from their current path very frequently as they approach moves they have not learned or can not apply easily.

In fact, Chu et al. [6] (on a similar NP-complete puzzle game, Sokoban) considers this deviation to be fairly complex and highly varied from person to person or skillset to skillset.

3.1.2 Field Solve Pathfinding

Recall the conditions for "flow" introduced in Section 2.2.1, namely the first condition demanding that "there is a balance between a user's perception of their skills and the offered challenge". In Minesweeper (and most other CSP games), players are allowed to control the subject of their own attention and are fairly unconstrained, hence players are free to alter their behaviour as they see fit, for example when a player believes the current sub-section of the field does not match their skill (most commonly when it exceeds their skill) and would prefer to be solving a portion of the field that more adequately maintains their flow and subsequently solve time.

There are many ways a player could alter their behaviour, such as their tendency to go for easy moves, their threshold for time sunk into a portion of a board before deviation, the tolerance for when to fall to a random guess, and ultimately their frustration level before ending the game and/or moving to a different field (to name a few). Players adapt their behaviour in accordance with the aforementioned flow, which they optimise in an effort to achieve their best solve time.

Our interpretation categorises such behavioural patterns down to "pathfinding behaviours" from the player asking themselves "where should I go" in response to "this place is not worth it". We suggest that the logical complexity, player skill, and navigational factors relevant in this thought process are sufficient to explain the end-to-end difficulty experienced by a player, and will explore such factors in depth shortly. However, because it is such a broad definition, this project can not reasonably test all such contributing factors in their entirety. Instead, since the logical aspect is already very well-researched, our eventual implementation and experiment will focus on the human skill aspect and eventually suggest future work that could account for field exploration.

3.1.3 "When Do I Leave"

The trigger for deviation in a player's solving process is when their ability to solve the "cluster" on the field does not meet the difficulty of the cluster.

Their ability to solve the cluster is defined by the following process: As previously explored by Pelánek [22]'s sudoku research, human CSP solving vaguely follows a particular protocol for how they apply their knowledge to play a game. Humans first start with the easiest techniques they are aware of and have identified in the field, which usually corresponds to techniques identifiable via RAC_1 such as Figure 5a or some more obvious RAC_2 moves such as Figure 5b.

After a human's known patterns are exhausted on a portion of the board, they may then fall back to their typical approach using constraint propagation to discern new patterns, where the player must enumerate possible states and progressively reduce them down to a reasonable solution. Alternatively, if the player has no known patterns whatsoever, they may also immediately go into this approach, a behaviour that's been observed by Hoffmann et al. [14] on participants approaching unseen CSP challenges.

3.1.4 "Where Do I Go"

Past this, upon reaching an area the player does not believe is worth solving, the player must choose the next area.

At or near the start of the game, the player's choices are purely random. A playthrough starts somewhere on the board with a random guess, and players continue randomly guessing until

they strike a cell with a hint 0, which subsequently triggers the auto-reveal feature of Minesweeper (recall Section 2.1).

Besides random guesses, there is are also more strategic guessing strategies. One such strategy is explored by Studholme [25] who suggests that, if the goal is to find a cell with a hint of 0, then it is optimal to pick the cell with the least neighbours. The reasoning comes from the probability of a cell being empty, which is itself given by the probability of a single neighbour containing a mine, raised to the power of the number of neighbours. Since the probability of a neighbour containing a mine is between 0 and 1, the probability of the chosen cell having a neighbour with a mine can only decrease as the number of neighbours increases.

Regardless of guessing strategy, the auto-revealed cells upon finding a 0-hint cell is then (usually) enough information for the player to start applying logical reasoning, learned heuristics, etc.

Among the newly revealed cells, the player can now either keep guessing to find more 0-hints (in hopes of getting a lucky extra boost) or begin solving the field via the process from Section 3.1.3.

Due to the sheer amount of options for guesses and direction for progression, there is a great deal of variance in this step that can lead to different difficulties, which is only amplified as player experience varies. Less experienced players with fewer learned patterns may identify less possible moves, and be more "starved" and hence gravitate strongly to the only moves they know. Conversely, more experienced players with a more accessible playing field, have more choice in where they move.

3.2 Compromises and Shortcuts On The Model

Many of the contributors to field difficulty which we've explored are either not necessary for a rough difficulty estimate, or can be sidestepped via some compromises to make implementation feasible or easier.

3.2.1 Pathfinding Compromises

With the extent to which player direction, guesses, and random decisions can influence the experienced difficulty for a playthrough, a proper difficulty ranking for a field should properly consider different paths and the variance in difficulty.

This could be computed directly by using a machine to enumerate every possible path and then concluding on the expected typical difficulty by processing each path individually. This is in practice of course very difficult because, for a 3BV value x (e.g. 40), there are x different moves that all have to be made at one point or another. For any two different paths on that field, these paths will both have the same moves (provided they're successful and efficient), just in a different order, hence there are $x!$ (e.g. $40! = 8.1591528 \cdot 10^{47}$) different possible paths.

One may think that there is no practical need to enumerate every such path, and indeed that is the case. consider the Minesweeper pattern shown in Figure 5b. We explained in section 2.6.1 how a player can "solve" this arrangement to progress in the field. Since this pattern is symmetric, the arbitrary decision (made in our explanation) to start with the north-most 2-hint cell can be flipped, solving the bottom half first and then progressing with the rest. Starting with the north square is effectively equivalent to the south square, provided that the player finishes solving the area. Starting with the north square (or the south square) and then proceeding elsewhere is NOT equivalent, and should definitely be treated as a separate path.

These paths are only subtly different in the whole scope of the playing field, and can be treated as practically equivalent with respect to perceived difficulty, allowing difficulty estimation to merge paths whose similarity metrics are within tolerance and skip the duplicates in enumeration, cutting down on the search space while still accounting for all paths in the final result.

These claims of equivalence assume there is some method for determining whether two paths can be considered the same. There are multiple ways the paths of a field can be quantised or checked for similarity, but the most straightforward is by the number of moves necessary to transform one path into the other, for example the two paths in the case above are equivalent after four moves.

Such a comparison is difficult to implement, since the conditions for checking path equivalence within distance of some amount of moves would be fairly complex to detect without false positives, for example a small cluster with difficult moves where one solve started with the easiest move and the other solve unknowingly started with the hardest (be it because they were rushing, inattentive, or otherwise).

Due to the level of attention and design that a proper path equivalence algorithm might take, instead of taking a field's expected path to be the average path over each possible solve of that field, weighted by its probability (i.e. the expected value of a random, correct, path through the field), we opted to use an empirical approach looking at only paths as played out by our records of humans solving the field. This avoids the need for path equivalency, since it directly produces paths proportional to their likelihood as sampled by actual players.

3.2.2 Skill Compromises

Player skill is entirely defined by the player's abilities. We do not have any method to directly learn the player's abilities, however we do have ways to directly measure a field's difficulty. As a workaround, we can observe how players behave upon encountering mine arrangements of different difficulties, and use this instead as a test of skill.

3.2.3 Logic Compromises

At the core of our perspective on field difficulty, we require a correct measure for the difficulty of a mine arrangement.

While sourcing an exhaustive list of Minesweeper techniques for use when analysing a Minesweeper playthrough would be helpful, such a list may be impossible to construct. Minesweeper patterns are not restricted to simply the 8 neighbours of a cell. Patterns are formed from different neighbour combinations across multiple cells, and can span the whole board in some cases. Since patterns are so unconstrained, a collection of every pattern may boil down to a naive collection of every possible assignment of mines to a field.

Instead of trying to identify every different type of move made, we can use the aforementioned Relational Arc Consistency (RAC) algorithm, which can be repeatedly applied while progressively increasing parameters, to identify the number of constraints that had to be considered for the move to be possible. This groups moves according to their logical complexity, and lets us look at Minesweeper moves in a purer form.

4 Implementation

We now move on from the overarching design and model for human difficulty to an implementation of RAC, and the platform which provides the supporting playthrough data for human-based path collection (in addition to data for use in evaluation).

4.1 Minesweeper Game

As we've resorted to using human playthroughs as sources for common paths within a field, human participant data is no longer solely for evaluation, but serves as part of our implementation as well.

Our aims necessitate data rich enough to recreate the individual moves made by the player, to permit looking at how the player solves the discrete instances of the Minesweeper Inference problem which comprise the full solve. Since there was no dataset fit for this purpose that was readily available, we collected playthroughs ourselves.

As was already covered by Jarus˘ek and Pelanek [15], a highly appropriate method for collecting such data is through the use of a self-administered experiment on a web-based experimental platform, where participants simply play the game at their leisure as if it was recreational play.

We designed a website (shown in Figure 6) featuring the appropriate experimental briefing and debriefing, and a reimplementation of Minesweeper using plain HTML5 canvas elements which limited the field pool to one of the pre-selected maps. The implementation included telemetry that tracked the type and timestamp of user input events, interpreted these as board actions, and then stored solve attempts in a mongoDB database upon completion, failure, or restart.

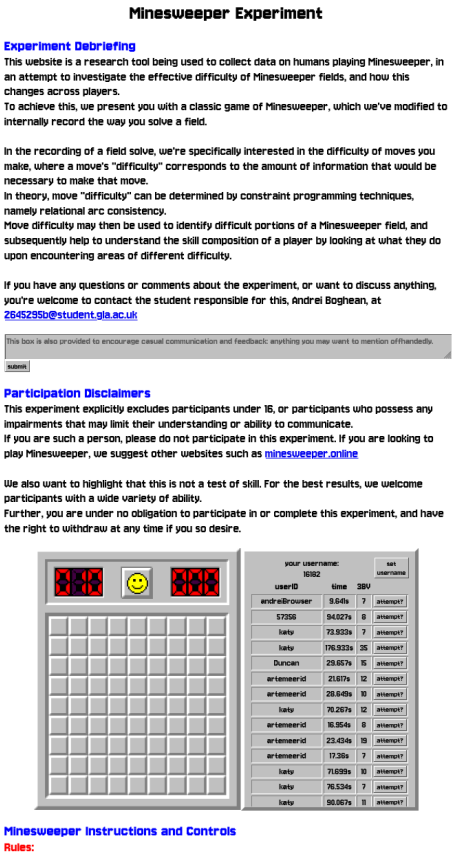


Figure 6: Playthrough collection and experiment website (https://minesweeper.andreiboghean.com)

4.2 Relational Arc Consistency

To process the dataset, a RAC implementation similar to the one demonstrated by Bayer et al. [2] was implemented to allow each move on each playthrough across every map to be annotated with the corresponding "difficulty", given simply by the m parameter necessary for RAC to deduce the move.

Before RAC could be used, the field PRNG seeds and move information had to be recovered and re-simulated locally to reproduce the original playthrough. The RAC implementation was then used to pre-process and cache the difficulty for every collected move, at which point the data was ready for analysis and evaluation.

5 Evaluation

Ideally, evaluation should test the whole process discussed for the user-experienced Minesweeper difficulty, however since our research stretched over a very broad range of possible factors, some factors are less well-defined than others.

In this evaluation, our main focus is on the core of the difficulty estimation, that being RAC and its suitability for predicting difficulty.

Separate to the main idea, we will also look at some auxiliary results which make use of RAC to provide other conclusions about a field's complexity or a player's ability, skill, playstyle, etc.

5.1 Experiment Setup

To evaluate whether our difficulty metric can correctly estimate the difficulty of a field to a degree satisfactory for a human player, we must compare our metric with an objectively correct ranking.

Existing Minesweeper datasets, such as the one published by Dhruv [11], do contain basic statistics such as solve time, 3BV, efficiency, etc. however no such dataset features any information to allow reproducing the fields used, meaning our difficulty estimator can not be applied. Further, there are no recordings of the move sequences used for included playthroughs, meaning our pathfinding compromises outlined in 3.2.1 also can not be applied. The most reasonable course of action was to produce our own dataset rich enough to enable both our difficulty estimation implementation and our subsequent evaluation, the implementation for which was further covered in Section 4.1.

We now proceed to define player skill and cell reward, two important definitions for the results and discussion that will follow.

5.1.1 Player Skill (for "auxiliary" results)

In investigating how player characteristics vary with player skill, we must start with defining what player skill is. This is difficult to define since not all players are playing with the same objective. Some players aim merely to complete the field, some players are aiming for speed, or some might be aiming to work on their technique rather than speed or even completion.

Since experienced Minesweeper players are typically aiming for solve time, we have decided to use solve time performance as a proxy for skill, under the assumption that player skill can be revealed through performance. For processing this ranking, we take a given player and we look at their percentage improvement with respect to the average time for a given field, and average

this across all fields. This aims to factor out different fields that take different times to solve by their nature (*e.g.* because they only take 2 clicks to solve).

5.1.2 Cell Reward

Continuing on the assumption that most players are aiming for solve time, we can also use this to define the "reward" the player is seeking throughout playing. With the importance we've attributed to the specific path that is taken through a field in solving, it follows that some cells may be more or less impactful in the solving process. The impact of a cell then determines both how rewarding it is for the player to solve it, and how important it is in the context of any playthrough within the field.

From the playthrough data we've collected, we define the reward for a cell on a field to be the average number of new moves the players were able to make upon solving the cell in question. For example, if the field had five possible moves in one state, and subsequently had seven possible moves after revealing the top-left cell, then our definition states the top-left cell has a reward of three, after one cell was consumed and three became solvable. Note this offsets the move actually made, so any cell which doesn't provide any info upon revealing will have a reward of zero (otherwise, the "reward" would technically be minus one).

5.2 Results

Our experimental analysis is split into 2 parts: field conclusions drawn from players, and player conclusions drawn from fields.

Field conclusions drawn from players make up our core results, and additional investigation on player behaviour is given in our auxiliary results.

Both field and player conclusions are viewed with respect to a ranking metric, where the metric is 3BV and player time advantage (as defined later in 5.1.1) respectively. For these categories, we look at a selection of statistics which are chosen intentionally to ensure specific phenomena can be caught in analysis (if they indeed occur).

5.2.1 Data composition

We received 987 raw submissions across 36 participants. After filtering, this was reduced to 789 submissions across 23 participants, 287 of which were complete solves of a field. Participants typically submitted approx 20 submissions, with some outliers returning to the experiment several times over multiple days producing submissions in excess of 50. The distribution of submissions per participant are shown in Figure 7. For reference, the solve times across submissions are also shown in Figure 8.

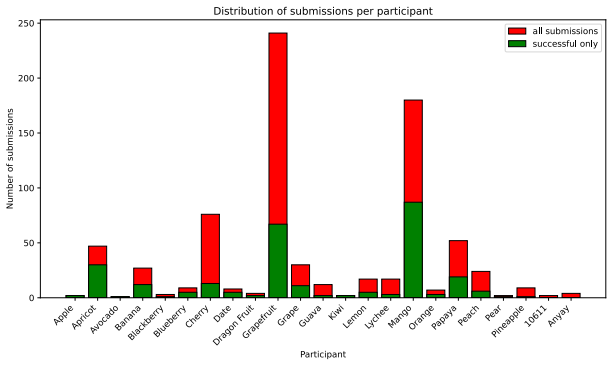


Figure 7: Submission distribution per participant ID

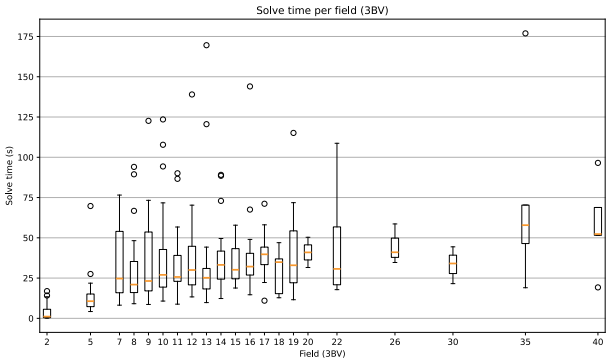


Figure 8: Violin plot of solve times per field

5.2.2 Primary Results

In Minesweeper field evaluation, the 3 core metrics we decided to look at (average solve time, summed cell average RAC difficulty, summed cell average reward) are listed in Figure 9, with the Pearson correlation matrix between these being available in Table 1.

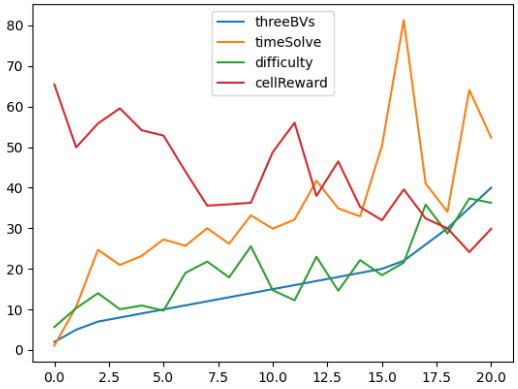


Figure 9: Measured core field statistics.

	3BV	Solve time	Difficulty	Cell reward
3BV	1.0*	0.75*	0.87*	-0.79*
Solve time		1.0*	0.66*	-0.66*
Difficulty			1.0*	-0.94*
Cell reward				1.0*

Table 1: Pearson correlations between core field statistics (* indicates $p \leq 0.05$)

5.2.3 Auxiliary Results

Existing Minesweeper datasets, such as the one published by Dhruv [11], already contain rich analyses on player click efficiency, solve speed, win distribution, etc. so we mostly stay away from these statistics in the interest of keeping focus on insights stemming from our specific use of RAC on individual moves (something not possible with existing datasets which did not collect individual moves). From review of submitted playthroughs

and other ad-hoc playing, we identified a selection of phenomena that are either broadly important or demonstrate nuanced but interesting behaviour. Behaviour which, if using a difficulty algorithm that is indeed suitable, should be discernible.

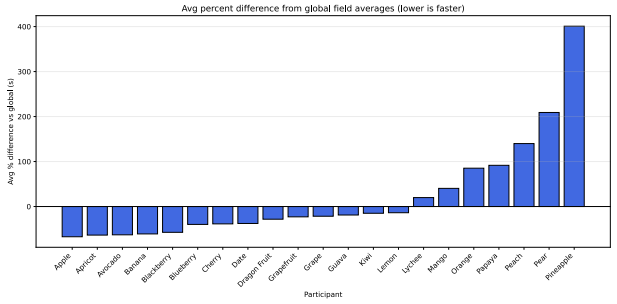


Figure 10: Avg. percent difference in field solve time per person

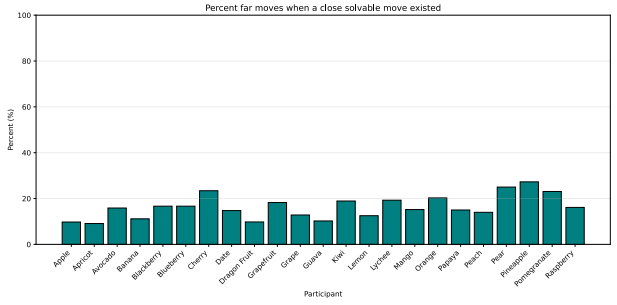


Figure 11: Percentage of moves that were unnecessarily far from the previous move

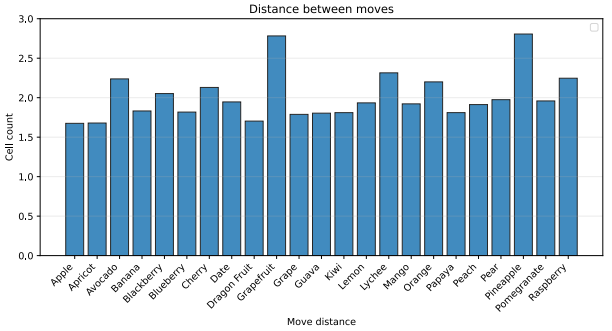


Figure 12: Avg. cell distance between all moves

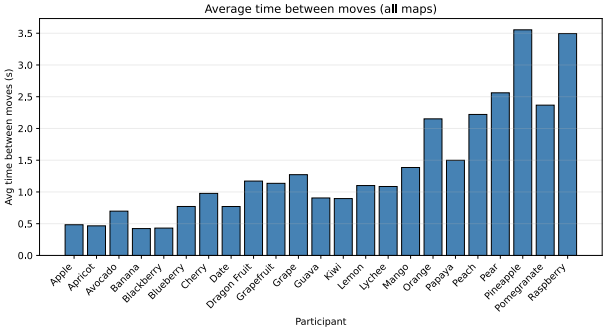


Figure 13: Avg. move time per person

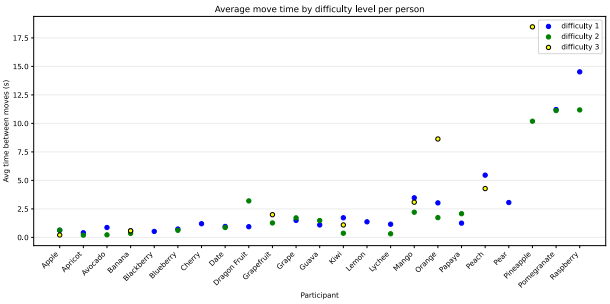


Figure 14: Move duration per RAC level per participant

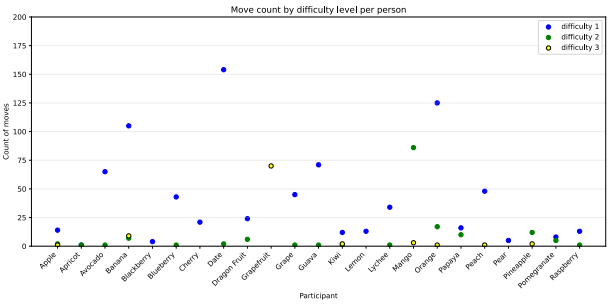


Figure 15: Move count per RAC level per participant

	Performance score (Figure 10)
% moves unnecessarily far (Figure 11)	0.45 (0.03)
Distance between moves (Figure 12)	0.33 (0.12)
Avg. time for a move (Figure 13)	0.95 ($\approx$ 0)

Table 2: Pearson correlations between auxiliary field statistics and player performance score (with p-value)

5.3 Discussion

In discussion, we cover the general insights that can or can not be made from our experimental results, whether these results are trustworthy and generalisable, and lay out what their wider implication is with respect to Minesweeper difficulty, player skill, or player pathfinding. We start with the primary discussion on RAC-informed difficulty, and then extend into auxiliary results using RAC to understand player skill and navigation.

5.3.1 Primary Discussion

Before our model for the difficulty of a Minesweeper field can be used to rank arbitrary Minesweeper fields without any human playthroughs, its core requires a suitable ranking metric for the individual moves within a playthrough. Our results here have found that summed RAC over the moves in a human playthrough on a field can produce a ranking that is strongly correlated with the measured average time to solve the field, which then suggests that summed RAC is fit for producing an objective, logical ranking for the difficulty of a field.

However, our results also find a strong correlation between time to solve with 3BV, and summed RAC with 3BV. These findings contradict the general belief in the Minesweeper community that 3BV is not sufficiently representative of the difficulty and solve time of a field, which indicates either that the community sentiment is incorrect or, more plausibly, our results are not generalisable to the Minesweeper fields which led the community to their conclusion.

There are three reasons why our results may not generalise: Firstly, the complaints in the community generally stem from players which are very involved and active in online social forums, players who are presumably highly competitive and play Minesweeper regularly with the aim to improve their skills. Our participants were mainly casual university students, many of whom had no prior experience and were learning the game throughout their participation, focusing purely on completion. The difference in goal, completion versus improvement, may then make a significant difference in the play style and subsequently the difficulty as measured by RAC. This was a symptom of our decision to use recorded playthroughs by human players instead of considering all possible paths, which meant the paths we used to inform difficulty were not indiscriminate, as a plain enumeration of every possible path could have been. Even with such an adjustment, the inverse of this problem may appear where difficulty focuses too much on experienced players rather than the broad population, or the focus could actually be too broad and include nonsensical paths.

Secondly, in a similar vein, our decision to evaluate beginner fields is not consistent with the most played field, that being Expert. Our decision to use the smallest field size was made not just to improve re-playability and encourage multiple submissions, but also in anticipation of computational difficulties with the larger field sizes. Larger field sizes may give rise to more complex interactions between areas of the field, and allow for more nuance and higher variance among boards with the same 3BV score.

Thirdly, our set of Minesweeper fields were not sufficiently broad. The field pool contained one map for 3BV values of 2, 5, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 22, 26, 30, 35, and 40. This selection did not feature the possibility for two maps to have the same 3BV but a significant difference in difficulty. Some adjacent fields with similar 3BV scores featured relatively drastic jumps in difficulty or time to solve, but we do not have the statistical significance to deny that this difference is from outlier fields.

5.3.2 Auxiliary Discussion

Our auxiliary results are broadly unrelated to the core results, which focus on the purely logical aspect of the difficulty in solving a Minesweeper field, instead looking at trends with respect to players and their "skill" as defined earlier in Section 5.1.1. They are still relevant to understanding how Minesweeper playing varies with respect to player ability, and fits in to the wider "pathfinding" aspect of our perspective on Minesweeper difficulty.

Our results begin with the simple idea that more experienced players are more efficient in their playing. We find a moderate positive correlation between the percentage of moves carried out by players which went unnecessarily far (see Figure 11), and the performance ranking of participants. This result suggests that players who are more experienced in Minesweeper find less need to deviate from their current field area in response to exceedingly difficult moves. While the p-value for this correlation is within tolerance (0.02), the absolute change in percentage is approx 10%, a relatively minor difference. While the correlation is logical and the statistical analysis technically confirms it, the correlation should be treated with caution. One possible explanation is that the actual range of ability was not varied enough among our participants, meaning only a small difference was observed because only a small difference was present. Another explanation is that the difficulties or structure of the evaluated fields did not benefit or punish missing a move, so there was never a need for players of any skill level to avoid missing a possible move. This explanation is more believable as it is consistent with our analysis of the core results, which suggests the fields used may not have been large enough to allow the complex logical structure we were anticipating.

Continuing to look at move trends, it is then tempting to investigate how the different difficulties of moves vary. In move difficulty time w.r.t. players (Figure 14), difficulty-2 moves are on par with difficulty-1 moves and even have a lower average time for some players. On the surface this is a surprising result since it suggests that the RAC difficulty does not impact the player's decision on whether to make the move, however when looked at in tandem with the frequency for these types of moves by the same participants (see Figure 15), it can be seen that difficulty 2 moves are used significantly less. This may lead to the conclusion that player performance is correlated with the variety of moves the player recognises and is able to play, as our results suggest less experienced players do not attempt to make any moves they do not immediately recognise as possible.

While this behaviour is suggested by our results, it can not be taken directly as a conclusion on human behaviour since both frequency per move difficulty and average time per move difficulty results have high variance. Instead, we highlight it to motivate the need in future work for specific RAC logging that can communicate the exact cells involved in a decision, so the relevant pattern can be identified and then recorded as being known and recognised by that player.

In a broader view of move trends, our results do show that the average move time (Figure 13) is highly correlated with player performance, but this result is as expected and generally uninteresting.

One trend that is interestingly weak in our findings is a weak positive correlation between average distance between moves, and player performance. This suggests that there is no correlation, which implies more experienced players do not see physically shorter paths or shortcuts as a result of their experience.

This implication makes sense in the context of Minesweeper, which is particularly unrestrictive in freedom of movement and allows the player to simply change their direction of travel rather than jump a great distance to an entirely different portion of the board. Doing this still has the drawback of consuming that easier path, and leaving the harder path for the player to eventually

solve in the future, eventually forcing the harder move if the player never finds any surrounding information that eases the complexity of the obstacle.

6 Conclusions

In conclusion, the results of our experiments show that CP techniques can directly estimate the solve time for a Minesweeper playthrough from the computed logical complexity, but our implementation cannot do so without processing the logical complexity from playthrough data which significantly limits its practical usefulness. Our experiments also yields results that contradict the widely believed opinion that field difficulty is not generally easily predictable, a conclusion which we believe can not be generalised beyond our collected dataset, due to insufficient coverage by our chosen fields.

In addition, our use of RAC as an analysis tool for human ability shows potential for quantifying a player's experience in the game, provided there is enough data on that player. We note, however, that it is not suitable in its current state, because our use of RAC only classified moves according to the number of constraints involved, which overlooks a significant degree of nuance that is present in Minesweeper moves even within the same number of constraints. On this topic we conclude that RAC, if modified to log the specific pattern of constraints, could be used to identify the Minesweeper technique used, and subsequently reveal the mastery a given player has over the various Minesweeper techniques.

7 Flaws and Future Work

In this section, we address the various compromises we have made in implementation and evaluation, and recap solutions or suggestions that would bring this project closer to a feasible difficulty estimation for arbitrary classical Minesweeper fields.

7.1 Alternatives to Human Playthroughs

The biggest flaw in our attempt to rank the difficulty of Minesweeper fields is in our decision to use the recorded human playthroughs as substitutes for the average path on the field.

On even a beginner 9x9 field with 9 mines, there are $\binom{81}{9} = 260887834350$ possible arrangements of mines, meaning Minesweeper fields are almost always a previously unseen generation. There is also very little interest by Minesweeper players to play the same fields, as any replay will naturally lead to learning order effects and diminishes the skill aspect. This is to say that it is unreasonable to rely on existing recordings of Minesweeper fields for a performance estimate. Finding the expected path is a very important step in proper difficulty prediction, which is very open with several methods that seem viable in theory, so a suitable alternative to averaging human paths is most important.

The first method, which we employed, is using existing playthroughs by other players. Our demonstrated method for sourcing the average path used statistical methods, treating the playthrough recordings as samples of the distribution of paths the player may follow when solving.

A first step in removing players may be using any alternative Minesweeper solver whether that be implemented via heuristics, complete algorithmic solutions, or even via machine learning. The paths found from computer solvers may overlap well with the paths humans may find in playing the same field, and may be sufficient to adequately predict the typical difficulty for a field, but it remains true that this is only sampling field paths, and may not be exhaustive. For a theoretically complete estimation, every path would need to be accounted for, likely cutting down the amount of enumeration necessary via some path equivalence algorithm. After cutting down, the pathfinding itself may be further split by considering pathfinding on different levels: first at the cluster level (see definition by Cicvárek [7]), then within the cluster.

7.2 Move Identification

$RAC_{(i,m)}$ only distinguishes Minesweeper moves as far as the number of constraints m and number of variables i needed to consider for a pattern to be recognised (recall that Generalised Arc Consistency is given by $RAC_{(1,1)}$, whereas RC_2 and RC_3 use an unbounded i, effectively meaning only the number of constraints is relevant beyond GAC). Within the same number of constraints, e.g. RAC_1 therefore, different types of moves that use the same difficulty but are a different pattern are treated as the same. Future work then should extend Relational Arc Consistency with some form of logging to assert which kind of pattern was used to make a conclusion about a cell. This information can then be used to record what specific type of patterns a person knows and is able to use, and ultimately build a better understanding of the player's capability to play Minesweeper.

In addition to this RAC extension, the breadth of moves considered could also be extended with some optimisations to allow using RAC beyond just 3 constraints without the severe time investment necessary in the current state. Alternatively, a MiniZinc model could be used as a catch-all for any moves beyond RAC3, which would also provide a method for identifying genuine guesses since any move made by the player not detectable by a formalised model running on a rigorous solver is almost certainly a guess.

7.3 Field Variety

While not necessary for a purely logical difficulty estimation, our augmentation of difficulty estimation with human playthroughs led to auxiliary investigation into player skill with respect to RAC difficulty, which is useful not only for informing the design of human-aware difficulty estimation, but also interesting for evaluating the composition of a Minesweeper player's skill-set.

Our experiments could not collect sufficient participants for the strength necessary to explore this avenue to it's full potential, so future work should focus on maintaining a set of good quality repeat participants, rather than one-off entries.

In addition, the fields used in study should also be of a higher variety, covering more fields within the same 3BV range, potentially at a larger size as well to allow for more variance and more complex interactions in field logic. This may however come at a cost on processing, that would need to be considered.

Lastly in the ream of field evaluation, the correlation between 3BV and field solve time should be formally proven or disproven, as the claim is currently anecdotal, being based on community consensus.

This is of particular importance, as the nature of 3BV and how it scales with fields may still prove useful in informing a better difficulty estimation, in spite of it not being a perfect estimation itself.

Acknowledgments

I would like to thank my project supervisor, Jessica Enright, for entertaining this overly-ambitious idea and tolerating my ramblings in our meetings. I'd also like to thank my friends and peers for their valuable insights, discussions, and general support throughout the lifetime of this project.

References

[1] Carlos Ansótegui, Ramón Béjar, Cèsar Fernández, Carla Gomes, and Carles Mateu. 2011. Generating highly balanced sudoku problems as hard problems. *Journal of Heuristics* 17, 5 (Oct. 2011), 589–614. https://doi.org/10.1007/s10732-010-9146-y

[2] Kenneth Bayer, Josh Snyder, and Berthe Y Choueiry. 2006. An Interactive Constraint-Based Approach to Minesweeper.

[3] David J. Becerra. 2015. Algorithmic Approaches to Playing Minesweeper. https://api.semanticscholar.org/CorpusID:3366717

[4] Stephan Bechtel. [n. d.]. *What is 3BV?* http://www.stephan-bechtel.de/3bv.htm

[5] cabbagery. 2020. Besides higher 3BV, what makes a board harder or easier? Reddit post. https://old.reddit.com/r/Minesweeper/comments/emq57m/besides_higher_3bv_what_makes_a_board_harder_or/fds870k/ Accessed: 23rd March 2026.

[6] Junyi Chu, Kristine Zheng, and Judith E Fan. 2025. What makes people think a puzzle is fun to solve?. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, Vol. 47.

[7] Jan Cicvárek. [n. d.]. Algorithms for Minesweeper Game Grid Generation. ([n. d.]).

[8] Jan Cicvárek and Štěpán Kopřiva. [n. d.]. Algorithms for Minesweeper Game Grid Generation. ([n. d.]).

[9] Ronny de Winter. 2009. 3BV Limits on Trial. (17 10 2009). https://minesweepergame.com/article/3bv-limits-on-trial-2009.pdf

[10] Rina Dechter. 2003. *Constraint Processing*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

[11] Kapur Dhruv. 2025. Minesweeper Exploratory Data Analysis. https://www.kaggle.com/code/kapurdhruv/minesweeper-eda/notebook

[12] Stefan Engeser and Falko Rheinberg. 2008. Flow, performance and moderators of challenge-skill balance. *Motivation and emotion* 32, 3 (2008), 158–172.

[13] Okihide Hikosaka, Shinya Yamamoto, Masaharu Yasuda, and Hyoung F. Kim. 2013. Why skill matters. *Trends in Cognitive Sciences* 17, 9 (2013), 434–441. https://doi.org/10.1016/j.tics.2013.07.001

[14] Ruth Hoffmann, Xu Zhu, Ozgur Akgun, and Miguel Nacenta. 2022. Understanding how people approach constraint modelling and solving. In *28th International conference on principles and practice of constraint programming (CP 2022) (Leibniz International Proceedings in Informatics (LIPIcs))*, Christine Solnon (Ed.). Schloss Dagstuhl- Leibniz-Zentrum fur Informatik GmbH, Dagstuhl Publishing. https://doi.org/10.4230/LIPIcs.CP.2022.28 Funding: This work is partially funded by NSERC Discovery Grant 2020-04401 (Canada). Xu Zhu: University of St Andrews and EPSRC grant DTG 1796157. ; 28th International Conference on Principles and Practice of Constraint Programming (CP 2022), CP 2022 ; Conference date: 31-07-2022 Through 05-08-2022.

[15] Petr Jarus˘ek and Radek Pelanek. [n. d.]. Difficulty Rating of Sokoban Puzzle. ([n. d.]).

[16] Richard Kaye. 2000. Minesweeper is NP-complete. *The Mathematical Intelligencer* 22, 2 (2000), 9–15.

[17] kendalltristan. 2021. What 3BV/s should a begginner, intermediate and expert player expect? Reddit post. https://www.reddit.com/r/Minesweeper/comments/lc7ux2/comment/glyiabr/?utm_source=share&utm_medium=web3x&utm_name=web3xcss&utm_term=1&utm_content=share_button Accessed: 23rd March 2026.

[18] Chanel J Larche and Mike J Dixon. 2020. The relationship between the skill-challenge balance, game expertise, flow and the urge to keep playing complex mobile games. *Journal of behavioral addictions* 9, 3 (2020), 606–616.

[19] LEBAldy2002. 2021. What defines the difficulty the levels? Reddit post. https://www.reddit.com/r/Minesweeper/comments/q854qd/comment/hgn5vlc/?utm_source=share&utm_medium=web3x&utm_name=web3xcss&utm_term=1&utm_content=share_button Accessed: 23rd March 2026.

[20] Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck, and Guido Tack. 2007. MiniZinc: Towards a Standard CP Modelling Language. In *Principles and Practice of Constraint Programming – CP 2007*, Christian Bessière (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 529–543.

[21] Kasper Allan Pedersen. 2004. The complexity of Minesweeper and strategies for game playing. https://api.semanticscholar.org/CorpusID:2440660

[22] Radek Pelánek. 2011. Difficulty Rating of Sudoku Puzzles by a Computational Model. In *The Florida AI Research Society*. https://api.semanticscholar.org/CorpusID:6431985

[23] Willian M. P. Reis, Levi H. S. Lelis, and Ya'akov Kobi Gal. 2015. Human computation for procedural content generation in platform games. In *2015 IEEE Conference on Computational Intelligence and Games (CIG)*. 99–106. https://doi.org/10.1109/CIG.2015.7317906

[24] Allan Scott, Ulrike Stege, and Iris Rooij. 2011. Minesweeper May Not Be NP-Complete but Is Hard Nonetheless. *The Mathematical Intelligencer* 33 (12 2011). https://doi.org/10.1007/s00283-011-9256-x

[25] Chris Studholme. 2001. Minesweeper as a Constraint Satisfaction Problem. https://api.semanticscholar.org/CorpusID:8838045

Appendices

evaluation ethics checklist

School of Computing Science
University of Glasgow

Ethics checklist form for 3rd/4th/5th year, and taught MSc projects

This form is only applicable for projects that use other people ('participants') for the collection of information, typically in getting comments about a system or a system design, getting information about how a system could be used, or evaluating a working system.

If no other people have been involved in the collection of information, then you do not need to complete this form.

If your evaluation does not comply with any one or more of the points below, please contact the Chair of the School of Computing Science Ethics Committee (matthew.chalmers@glasgow.ac.uk) for advice.

If your evaluation does comply with all the points below, please sign this form and submit it with your project.

---

1. Participants were not exposed to any risks greater than those encountered in their normal working life.

   Investigators have a responsibility to protect participants from physical and mental harm during the investigation. The risk of harm must be no greater than in ordinary life. Areas of potential risk that require ethical approval include, but are not limited to, investigations that occur outside usual laboratory areas, or that require participant mobility (e.g. walking, running, use of public transport), unusual or repetitive activity or movement, that use sensory deprivation (e.g. ear plugs or blindfolds), bright or flashing lights, loud or disorienting noises, smell, taste, vibration, or force feedback

2. The experimental materials were paper-based, or comprised software running on standard hardware.

   Participants should not be exposed to any risks associated with the use of non-standard equipment: anything other than pen-and-paper, standard PCs, laptops, iPads, mobile phones and common hand-held devices is considered non-standard.

3. All participants explicitly stated that they agreed to take part, and that their data could be used in the project.

   If the results of the evaluation are likely to be used beyond the term of the project (for example, the software is to be deployed, or the data is to be published), then signed consent is necessary. A separate consent form should be signed by each participant.

   Otherwise, verbal consent is sufficient, and should be explicitly requested in the introductory script.

4. No incentives were offered to the participants.

   The payment of participants must not be used to induce them to risk harm beyond that which they risk without payment in their normal lifestyle.

5. No information about the evaluation or materials was intentionally withheld from the participants.
   Withholding information or misleading participants is unacceptable if participants are likely to object or show unease when debriefed.

6. No participant was under the age of 16.
   Parental consent is required for participants under the age of 16.

7. No participant has an impairment that may limit their understanding or communication.
   Additional consent is required for participants with impairments.

8. Neither I nor my supervisor is in a position of authority or influence over any of the participants.
   A position of authority or influence over any participant must not be allowed to pressurise participants to take part in, or remain in, any experiment.

9. All participants were informed that they could withdraw at any time.
   All participants have the right to withdraw at any time during the investigation. They should be told this in the introductory script.

10. All participants have been informed of my contact details.
    All participants must be able to contact the investigator after the investigation. They should be given the details of both student and module co-ordinator or supervisor as part of the debriefing.

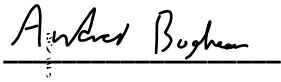
11. The evaluation was discussed with all the participants at the end of the session, and all participants had the opportunity to ask questions.
    The student must provide the participants with sufficient information in the debriefing to enable them to understand the nature of the investigation. In cases where remote participants may withdraw from the experiment early and it is not possible to debrief them, the fact that doing so will result in their not being debriefed should be mentioned in the introductory text.

12. All the data collected from the participants is stored in an anonymous form.
    All participant data (hard-copy and soft-copy) should be stored securely, and in anonymous form.

Project title Difficulty Breakdown of the Human Playstyle in Minesweeper

Student's Name Andrei Boghean

Student Number 2645295b

Student's Signature 

Supervisor's Signature 

Date 23/1/2026